

Assignment 3: Activity Recognition (20P)

Hand in in groups of two.

Goals

You are familiar with the basic principles of machine learning. You can implement a simple machine learning classifier to detect activities based on real-time sensor data.

1 Gathering Training Data

6P

Recommended Packages: DIPPID, pandas

Create a program called *gather_data.py* which captures sensor data (accelerometer and gyroscope) from the DIPPID device and saves it to a CSV file. This sensor data will be used to train a machine learning classifier later.

Record data sets for each of the following activities: *running, rowing, lifting, jumpingjacks*. A depiction of those activities can be seen in Figure 1. To add some variety in data and capture a big enough data set,

- vary sampling rate: 20 Hz and 100 Hz
- vary placement of DIPPID device: in hand and in pocket of pants
- record at least 5 data sets for each activity, sample rate, and placement ($4 \times 2 \times 2 \times 5 = 80$)
- for each recording, perform the activity for at least 10 seconds

It should be possible to use the DIPPID device to start capturing data. Data logging stops automatically when a fixed time has passed. Each recorded data set is saved separately to a CSV file. The CSV files should be structured as follows:

File Name: *your_name-activity-sample_rate-placement_number.csv*
(e.g., *susi-running-20Hz-pocket-1.csv*)

Columns: *id, timestamp, acc_x, acc_y, acc_z, gyro_x, gyro_y, gyro_z*

In addition to including the data in your own repository, upload it to a repository shared with others (see GRIPS). This way, all of you can use the whole data set for Task 2.

Uploading your data is due to May 13, 1pm to ensure that all of you have access to the train data in time. Upload is required per person, not per group.

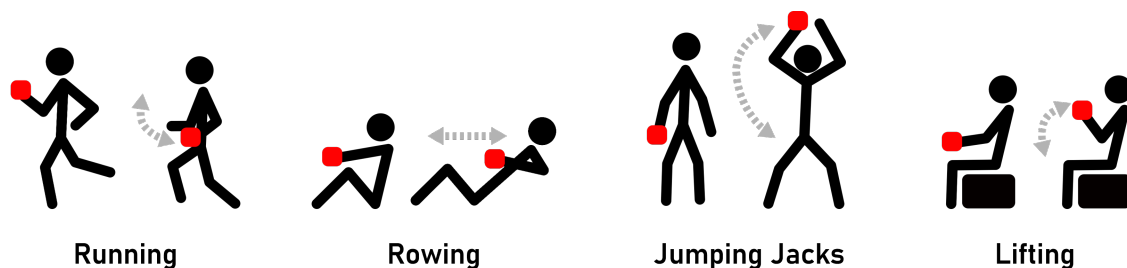


Figure 1: Four different fitness activities.

Score

- (1P) data is logged correctly
- (1P) log files are named and structured appropriately
- (1P) logging can be started with the DIPPID device
- (1P) sampling rate was varied
- (1P) placement was varied
- (1P) enough data sets captured

2 Activity Recognition

14P

Recommended Packages: *pandas, numpy, pyglet, sklearn (scikit-learn)*

Create a program called *fitness_trainer.py*. The application should help users with their fitness training by displaying activities and tracking whether they were executed correctly for a certain amount of time.

But first, you have to train and evaluate a classifier: read training data from CSV files, split them into a train and test data set, and train a machine learning classifier. Most certainly, it is required to pre-process training data before training (e.g. filtering, normalization, transformation into frequency domain, ...).

After the training has finished, evaluate your model's accuracy using the test data set. Systematically compare **kernel functions**, **sampling rates**, and **input parameters and pre-processing**. Document decision process, procedure, tested options, and results. Extend your documentation with meaningful plots.

For implementing the fitness trainer, use the model with best results in your evaluation. The program should predict activities based on sensor data from the DIPPID device. The prediction should run continuously without requiring further intervention by the user. Visualize the fitness trainer nicely using pyglet.

Score

- (1P) reasonable train-test-split
- (1P) comparison of kernel functions
- (1P) comparison of sampling rates
- (2P) comparison of input parameters and pre-processing
- (2P) overall documentation and plots

- (1P) the "best" classifier is trained as soon as the fitness trainer program is started
- (3P) the fitness trainer classifier recognizes activities correctly
- (2P) prediction works continuously without requiring intervention by the user
- (1P) the fitness training application works and looks nice

Code Quality: bad code quality and missing documentation can lead to loss of points.